

HW04 — Automation Testing on EShop · Báo cáo chính

Sinh viên: Phạm Vũ Ngọc Duy — **MSSV:** 23127183 **SUT:** EShop — <https://github.com/ttbhanh/eshop-sut> **Repo bài làm:** <https://github.com/DuyPham111/HW04> **Công cụ:** Playwright 1.62.1 + Playwright HTML reporter · Node v22.16.0 · Windows 11 **AI:** Claude Code (Sonnet 5 / Opus 5) — log đầy đủ ở [ai-audit/ai-audit-report.md](#) **Ngày chạy chính thức:** 2026-08-16 **Video demo Task 2:** <ĐIỀN SAU KHI QUAY>

0. Tổng quan

0.1 Ba feature (\$5)

Lấy lại **đúng 3 feature web đã chọn ở HW02**, mỗi Pool một feature. Pool D (mobile) **không dùng** ở HW04 vì đề \$5 nói rõ bài này automation **web frontend**.

	Pool	FR	Feature	App / route	Data file
A	A	FR-02	Đăng nhập & Khóa tài khoản	web :5173 /login	feature-a-login.csv
B	B	FR-09	Mã giảm giá (Coupon)	web :5173 /checkout	feature-b-coupon.json
C	C	FR-15	Quản lý Sản phẩm (CRUD)	admin :5174 tab Sản phẩm	feature-c-product-admin.csv

Không trùng feature trong nhóm: ba feature này **không đổi một chút nào** so với HW02, nên thỏa thuận phân công không trùng mà nhóm đã chốt ở HW02 vẫn còn nguyên hiệu lực — HW04 không phát sinh lựa chọn mới nào cần đăng ký lại.

0.2 Số liệu tổng

Mọi con số dưới đây **copy từ** [reports/summary.md](#), sinh tự động bằng `node tools/summarize.mjs` đọc từ 9 file JSON kết quả thật. Không con số nào đếm tay.

Chỉ số	Giá trị
Số feature automation	3
Số test case automation	53 (A: 16 · B: 18 · C: 19) — vượt mức ≥ 12 /feature của \$6
Số lượt chạy browser	9 (3 feature × chromium/firefox/webkit)
Số lần thực thi (TC × engine)	159
Pass	76
Fail	83
Flaky	0
Skipped	0
Test case Fail ở ≥ 1 engine	29

Số defect truy được	16 — 2 bug MỚI + 14 defect đã ghi từ HW02
Tổng thời gian chạy	881.1s

Vì sao Fail nhiều lại là đúng: kỳ vọng trong mọi file dữ liệu lấy theo **đặc tả FR** (`eshop-sut/README.md`), không theo hành vi hiện tại của SUT. SUT này được thiết kế có bug cố ý, nên Fail = SUT lệch đặc tả. Nếu điền kỳ vọng theo hành vi hiện tại thì suite Pass 100% và **không phát hiện được bug nào**.

83 lần Fail → 29 test case → 16 defect. Đọc số Fail như đếm bug là sai gần 6 lần: riêng B001 (bộ đếm khóa) gây 5 TC × 3 engine = 15 lần Fail nhưng vẫn chỉ là 1 defect. Bảng quy đổi đầy đủ ở [bug-report/bug-report.md](#) §0.

1. Chiến lược automation

1.1 Quy trình dùng AI theo từng bước (§2)

§2 cấm ra một prompt gộp kiểu "viết toàn bộ automation script cho feature X". Với **mỗi** feature, quy trình đi đúng 6 bước, mỗi bước một lượt hỏi và **người duyệt trước khi qua bước sau**:

Bước	Việc	Kết quả kiểm chứng được
1	Đọc UI thật — selector khả dụng · state nằm đâu · điều hướng · bug chặn đường	Bảng "sự thật DOM" trong comment đầu mỗi page object
2	Chuyển test case HW02 → file dữ liệu ngoài .csv/.json	53 dòng dữ liệu, mỗi dòng truy về đúng 1 TC của HW02
3	Chốt assertion pattern cho từng TC trước khi sinh code	Cột technique + bảng §1.3
4	Page object trước (chỉ selector + hành động, không assertion)	5 file trong tests/pages/
5	Spec sau (chỉ vòng lặp + assertion)	3 file .spec.js, grep không ra test data
6	Chạy thật → phân loại từng Fail → sửa	§3

Điểm đáng ghi nhất về quy trình: vì bước 1 (đọc UI thật) được **tách riêng và làm trước**, selector của cả 3 feature **đúng ngay từ lần sinh đầu** — không có vòng "AI đoán selector sai → gãy → sửa" mà quy trình thông thường hay gặp. Đối lại, các lỗi thực sự xảy ra đều thuộc loại **tinh vi hơn**: sai ở tầng dữ liệu, tầng công cụ, hoặc ở giả định về môi trường vận hành (§3.2).

1.2 Data-driven (§6)

§6: "The test data must be stored in a separate .csv or .json file — hardcoded inline arrays or objects in the script are not accepted."

- **Toàn bộ** test data nằm ở `tests/data/` . Dùng **cả hai định dạng** để chứng minh loader xử lý được cả hai: `.csv` (Feature A, C) và `.json` (Feature B).
- Spec file chỉ `loadCsv()` / `loadJson()` rồi lặp. Tự kiểm bằng:

```
grep -inE "eshop\.com|SAVE10|BIGBUY|VIP100|Test1234|Admin123" tests/*.spec.js → 0 kết quả
```

- **Token** trong ô dữ liệu, giải ở tầng loader để file vẫn đọc được bằng mắt và bằng Excel:

Token	Nghĩa	Vì sao cần
<empty>	chuỗi rỗng	CSV không diễn tả được ô rỗng khác ô thiếu
<spaces:N>	N khoảng trắng	test "tên toàn khoảng trắng" (B010)
<repeat:X:N>	X lặp N lần	biên độ dài 255/256 ký tự (B015)
<uniq>	mã duy nhất theo lần chạy	user/sản phẩm không đựng dữ liệu lần chạy trước

- **Nguyên tắc quan trọng nhất:** cột `expect` điền theo **ĐẶC TẢ**, không theo hành vi SUT. Đây là lý do suite phát hiện được bug thay vì hợp thức hoá chúng.

1.3 Assertion pattern (§6 đòi ≥3 pattern khác nhau về bản chất)

Suite dùng **5 pattern**, khác nhau ở **thứ được kiểm**, không phải khác cú pháp:

#	Pattern	Kiểm cái gì	Bắt được lớp bug nào	A	B	C
P1	DOM / web-first	trạng thái thấy được trên UI	thiếu thông báo lỗi, sai nhãn, mật khẩu không che	✓	✓	✓
P2	Navigation / URL	có điều hướng đúng nơi spec nói không	đăng nhập thành công mà không rời /login	✓	—	—
P3	Backend state (REST)	state thật trong DB	UI báo lỗi mà DB vẫn đổi, và ngược lại	✓	✓	✓
P4	Soft numeric	tiền, bộ đếm, độ dài	công thức % sai, bộ đếm +2, tên 256 ký tự	✓	✓	✓
P5	Network / HTTP status	request có gửi không, server trả mã gì	chặn sai tầng; 401 vs 403; thiếu xác thực	✓	✓	✓

Ba điều cần nói rõ:

1. `toBeVisible` / `toContainText` / `toHaveText` là **ba biến thể cú pháp của cùng một pattern (P1)**, chỉ tính là **một**. Đếm chúng thành 3 pattern là hiểu sai yêu cầu §6.
2. **Feature B không dùng được P2** — thanh toán thành công **không đổi URL** (`Checkout.jsx` chỉ đổi state `success` rồi render lại trong cùng route). Bù bằng **P3** (đối chiếu số trên UI với `body API` thật).
3. **Feature C cũng không dùng được P2** — admin là SPA một route duy nhất, chuyển tab bằng React state nên URL luôn là `http://localhost:5174/`. Bù bằng **P3** cho mọi TC.

Vì sao P5 là pattern quyết định của Feature A: `Login.jsx` bắt mọi lỗi rồi hiển thị **một câu duy nhất** "Đăng nhập thất bại. Vui lòng kiểm tra lại." — nên trên UI, "sai mật khẩu" (401) và "tài khoản đang bị khóa" (403) **trông y hệt nhau**. Assertion trên DOM một mình **không thể** phân biệt hai tình huống này. Chỉ P5 (đọc HTTP status) và P3 (đọc `login_attempts` / `locked_until` trong DB) mới tách được ⇒ đây là bằng chứng cụ thể cho luận điểm "assertion trên UI là không đủ".

1.4 Cô lập dữ liệu và tính lặp lại

Cả 3 feature đều đựng state dùng chung trong SQLite. Không cô lập thì 9 lượt chạy sẽ đạp chân nhau và kết quả vô nghĩa làm bằng chứng.

Feature	Rủi ro	Cách cô lập
A	login_attempts là state DB, khóa kéo dài 180 giây	Mỗi TC khóa tài khoản dùng user dùng-một-lần tạo qua POST /api/register (lock01-<uniq>@hw04.test), dọn qua DELETE /api/admin/users/:id
B	Giỏ hàng nằm trong React state (mất khi reload); coupon_usage không có API xóa	Điều hướng SPA trong một page session ; TC giới hạn lượt dùng user mới mỗi lần
B	Đăng nhập qua form sẽ dính chính cơ chế khóa của FR-02	Nhét token vào localStorage bằng addInitScript trước khi trang tải
C	CRUD sinh dữ liệu thật	Tên sản phẩm luôn có tiền tố HW04- + <uniq>; dọn qua API sau mỗi test

Bảo vệ dữ liệu seed: SEED_EMAILS trong tests/utils/env.js là **danh sách bảo vệ** — bước dọn dữ liệu không bao giờ được xóa admin@eshop.com / test@eshop.com .

Kết quả kiểm chứng: sau **cả 9 lượt**, DB còn **đúng 5 sản phẩm seed và 2 tài khoản seed**, không sót một bản ghi rác nào. **0 flaky** trên 159 lần thực thi.

2. Kết quả thực thi

2.1 Chín lượt browser

#	Feature	Engine	Test	Pass	Fail	Flaky	Report
1	A	chromium	16	6	10	0	reports/html/a-chromium/
2	A	firefox	16	6	10	0	reports/html/a-firefox/
3	A	webkit	16	6	10	0	reports/html/a-webkit/
4	B	chromium	18	11	7	0	reports/html/b-chromium/
5	B	firefox	18	9	9	0	reports/html/b-firefox/
6	B	webkit	18	11	7	0	reports/html/b-webkit/
7	C	chromium	19	9	10	0	reports/html/c-chromium/
8	C	firefox	19	9	10	0	reports/html/c-firefox/
9	C	webkit	19	9	10	0	reports/html/c-webkit/

Mỗi lượt là **một lệnh playwright test riêng**, ghi vào **một thư mục report riêng** — không phải một report gộp.
Lý do kỹ thuật: Playwright **xóa sạch outputDir** ở đầu mỗi lần chạy, nên nếu 9 lượt dùng chung thư mục thì ảnh Fail của 8 lượt đầu bị mất.

2.2 Bảng chứng Run by: 23127183 (\$6, \$11)

MSSV + timestamp ISO xuất hiện ở **ba chỗ độc lập** trên mỗi report, để TA kiểm ở chỗ nào cũng thấy:

#	Chỗ	Do đâu sinh ra
1	Khối metadata đầu report	playwright.config.js → metadata
2	Annotation trên từng test case	fixture runMeta (auto: true) trong tests/fixtures/base.js
3	Dài cố định chân trang + thẻ <title>	tools/stamp-report.mjs, đọc số thật từ file JSON của đúng lượt đó

Ảnh bằng chứng: 19 ảnh trong [reports/evidence/](#) — 9 ảnh toàn cảnh report, 9 ảnh cận cảnh dài Run by, 1 ảnh chi tiết test case Fail.

Ví dụ nội dung dài Run by của lượt a-firefox :

```
Run by: 23127183 – Phạm Vũ Ngọc Duy | Run at (ISO 8601): 2026-08-16T11:55:27.671Z
| Feature A – FR-02 Đăng nhập & Khóa tài khoản · firefox | 16 test · 6 pass · 10 fail · 171.0s

tools/stamp-report.mjs không bịa số: mọi con số trong dài đọc từ file JSON kết quả thật của chính lượt đó.
Script chỉ trình bày lại.
```

2.3 Khác biệt giữa 3 engine · số flaky

0 flaky trên 159 lần thực thi. Phân bố kết quả theo engine:

Kiểu	Số TC	Ý nghĩa
Fail giống hệt trên cả 3/3 engine	27	Bằng chứng rất mạnh: Fail phản ánh bug của SUT , không phụ thuộc trình duyệt hay timing
Fail chỉ trên 1 engine	2	Đã điều tra — không phải bug SUT , xem bên dưới

Điều tra 2 TC lệch: FR09-DT-02 và FR09-DT-03 chỉ Fail trên **firefox**. Đọc thông báo lỗi thật:

```
Error: browserContext.close: Protocol error (Browser.removeBrowserContext):
can't access property "_maybeDontRestoreTabs", this._windows[aWindow.__SSI] is undefined
```

Đây là lỗi **hạ tầng của chính Firefox** lúc đóng browser context sau khi chạy suite dài — **không** phải bug SUT, cũng **không** phải flaky wait trong script (assertion nghiệp vụ đã chạy xong và **Pass** trước khi lỗi này xảy ra). **Đã chạy lại riêng 2 TC này trên firefox → PASS.**

```
npx playwright test tests/feature-b-coupon.spec.js --project=firefox -g "DT-02|DT-03"
```

⇒ Xếp vào nhóm **hạn chế môi trường**, không báo là bug (xem bug-report.md §3).

3. Human review & Gap analysis

3.1 Phân loại toàn bộ 29 test case Fail

Nhóm	Số TC	Xử lý
Bug thật — HW02 đã ghi	23	Ghi vào bug-report.md §2, nối tới Issue cũ của repo nhóm

Bug thật — MỚI ở HW04	4 → 2 defect	Ghi đầy đủ + tạo GitHub Issue kèm ảnh
Script sai	0 còn lại	12 lỗi đã tìm & sửa trước khi chạy chính thức — xem §3.2
Pass giả	0 còn lại	1 Pass giả đã phát hiện & siết lại — xem §3.3
Hạn chế môi trường	2	FR09-DT-02, FR09-DT-03 — lỗi hạ tầng Firefox (§2.3)
	29	

3.2 AI sai / sót cái gì, và vì sao — 12 lỗi thật

Đây là danh sách các lỗi **đã thực sự xảy ra** trong bài này, không phải danh sách lý thuyết. Mỗi dòng đều dẫn được về commit đã sửa nó.

#	AI sai / sót cái gì	Vì sao AI sót	Tôi đã sửa thế nào
1	parseMoney nuốt dấu âm	Đặc thù feature — hàm viết khi làm Feature A, vốn không có phép tính tiền nào, nên chỉ được kiểm với số dương. Bug B007 sinh ra tiền giảm âm (-54.000.000 đ) mà hàm trả về 54000000 ⇒ test vẫn Fail nhưng bug report sẽ ghi sai trị số thực tế	Giữ dấu trừ khi nó nằm ngay trước cụm chữ số; unit-test 8 trường hợp
2	loadJson tự trim chuỗi → Pass giả	Giới hạn mô hình — áp dụng quy tắc hợp lý của CSV (khoảng trắng quanh ô là nhiều định dạng) sang JSON mà không phân biệt ngữ cảnh. Trong JSON, " SAVE10 " là khoảng trắng cố ý	resolveToken(raw, { trim }); CSV giữ true, JSON dùng false — xem §3.3
3	Thiếu cách so khớp text tiếng Việt không phụ thuộc dấu	Chất lượng prompt — tôi yêu cầu ghi expectedError dạng không dấu (cho an toàn bảng mã) nhưng không nói rõ phía so khớp cũng phải bỏ dấu	Thêm normalizeVi() + expectVisibleTextVi(), unit-test với 4 thông báo thật của SUT
4	File CSV thiếu BOM UTF-8	Giới hạn mô hình — ghi UTF-8 thuần là mặc định của mọi công cụ, tối ưu cho máy đọc (fs.readFileSync không cần BOM). AI không mô phỏng bước " <i>người dùng mở file này bằng Excel trên Windows</i> " — nơi thiếu BOM làm hỏng toàn bộ dấu tiếng Việt	Ghi lại file kèm BOM + thêm stripBom() để loader không vỡ
5	loadCsv bỏ sót dòng comment bị Excel quote	Giả định môi trường vận hành — bộ lọc kiểm # trên chuỗi thô , ngầm giả định " <i>dòng comment không bao giờ bị CSV-quote</i> ". Đúng với file tự ghi, sai khi file đi qua một vòng Excel "mở → sửa → lưu"	Tách ô bằng splitCsvLine(1)[0] trước rồi mới kiểm #
6	page.goto('/') vào nhầm app (web :5173)	Đặc thù feature — baseUrl trong config trỏ vào web (đúng cho Feature	Dùng URL tuyệt đối page.goto(ADMIN_URL)

	thay vì admin :5174)	A/B). Feature C là app thứ hai, khác port — trường hợp đầu tiên trong bài mà baseUrl không còn đúng	
7	Cột categoryName ghi "Phu kien" (không dấu) trong khi UI thật là "Phụ kiện"	Chất lượng prompt / quy ước — tôi đặt quy ước "viết không dấu" cho phần ghi chú , AI áp nhầm sang cả cột dữ liệu chức năng . selectOption({label}) so khớp chính xác nên luôn thất bại	Sửa 15/15 chỗ về đúng có dấu
8	fillProduct({name, ...}) sai tên khoá — cột CSV là productName	Giới hạn mô hình — name là tên tự nhiên hơn cho tham số, AI viết theo thói quen thay vì theo schema dữ liệu đã định. Hậu quả: ô "Tên sản phẩm" không bao giờ được điền	Đổi tham số thành productName khớp đúng tên cột
9	Ảnh chụp lúc Fail trắng tinh ở 3 TC SEC-*	Đặc thù feature — 3 TC này là test thuần API , page chưa từng điều hướng đi đâu, nên screenshot: only-on-failure chụp đúng about:blank. AI viết test API mà quên cơ chế chụp ảnh của Playwright gắn với page	Thêm page.goto(API_URL + '/api/products') sau khi đọc xong status ⇒ ảnh hiện đúng JSON thật (<i>lỗi này do người dùng phát hiện, không phải AI tự tìm</i>)
10	BV-R03 bị trộn lẫn với chính bug FR-12	Giới hạn mô hình — AI viết comment nêu đúng ý định ("BV-R03 dùng token, khác SEC-") rồi không cài đặt ý định đó trong code; mọi dòng mode=security đều dùng context không token. TC kiểm ràng buộc FK bị chặn bởi bug thiếu auth trước khi chạm tới nhánh cần kiểm ⇒ vi phạm single-fault	Thêm isAuthBypassTest; BV-R03 dùng token hợp lệ
11	Tự thêm --reporter=list vào CLI → mất toàn bộ HTML/JSON report	Giới hạn mô hình — chính playwright.config.js mà AI đã viết có sẵn comment cảnh báo đúng vấn đề này , nhưng khi thêm cờ ở bước sau, AI không đối chiếu lại với comment của chính mình	Bỏ --reporter khỏi CLI; đã viết 2 thí nghiệm đối chứng để xác định cơ chế trước khi kết luận
12	Cột Engine luôn ghi "chromium" + runLabel bị cắt cụt	Hai nguyên nhân độc lập: (a) report.config.projects liệt kê cả 3 project cấu hình , không phải project đã chạy — đọc projects[0] luôn ra phần tử đầu; (b) đặc thù nền tảng — spawnSync(..., {shell:true}) trên Windows không tự quote arg, nên chuỗi có khoảng trắng bị tách vụn	(a) đọc spec.tests[0].projectName; (b) thêm quoteArg(). Không chạy lại test vì JSON gốc vẫn đúng, chỉ đóng dấu lại report

Nhận xét về phân bố nguyên nhân: 0/12 lỗi là "selector sai" — loại lỗi kinh điển nhất khi để AI sinh test. Lý do là **bước 1 (đọc UI thật) được tách riêng và làm trước** cho cả 3 feature. Đổi lại, 12 lỗi thực tế đều nằm ở tầng sâu hơn: **dữ liệu** (1,2,3,4,5,7), **công cụ** (11,12), **giả định về môi trường** (4,5,6,12b), và **khoảng cách giữa ý định đã viết ra và code thật sự chạy** (10,11).

3.3 Pass giả đã phát hiện và siết lại

TC FR09-BV-R03 — "Robust: mã giảm giá có khoảng trắng thừa ' SAVE10 ' "

- **Mục đích TC:** kiểm xem **SUT** có tự cắt khoảng trắng thừa trước khi gửi mã lên API không.
- **Tại sao là Pass giả:** `resolveToken()` của loader gọi `.trim()` trên **mọi** giá trị. Chuỗi `" SAVE10 "` trong file JSON bị **loader cắt sạch khoảng trắng trước khi tới tay test**. Test nhận vào `"SAVE10"` đã sạch, gửi lên, được chấp nhận, và **báo Pass** — nhưng **công lao trim là của loader, không phải của SUT**. TC không hề kiểm được điều nó tuyên bố.
- **Phát hiện thế nào:** không phải bằng cách chạy test (test vẫn xanh), mà bằng cách **đọc lại đường đi của dữ liệu** khi soạn file JSON cho Feature B — nhận ra `resolveToken` nằm giữa file dữ liệu và test.
- **Đã siết lại:** `resolveToken(raw, { trim })` — CSV giữ `trim: true` (khoảng trắng quanh ô là nhiều định dạng), **JSON dùng `trim: false`** (khoảng trắng trong JSON là cố ý). Xác minh:

R03 couponCode: " SAVE10 " — độ dài 10 ← trước khi sửa: "SAVE10", độ dài 6

- **Kết quả sau khi siết:** TC vẫn Pass, nhưng **giờ Pass vì lý do đúng** — client của SUT thật sự gọi `.trim()` trước khi gửi (`Checkout.jsx: couponCode.trim().toUpperCase()`).
- **Kiểm chứng không hồi quy:** Feature A (dùng CSV) vẫn đọc đúng 16 dòng sau thay đổi.

Bài học: Pass giả nguy hiểm hơn Fail sai, vì Fail thì bị điều tra còn Pass thì được tin. Loại Pass giả khó thấy nhất là khi **chính họ tăng test đã âm thầm làm họ việc mà SUT đáng lẽ phải làm**.

3.4 Cải tiến prompt rút ra

Prompt cũ	Vấn đề gặp phải	Prompt cải thiện
"Viết hàm đọc số tiền hiển thị"	Chỉ đúng với số dương (lỗi #1)	"...Liệt kê trước các dạng giá trị hàm sẽ gặp (âm, NaN, nhiều định dạng phân cách nghìn), viết unit-test cho từng dạng, rồi mới cài đặt."
"Giải token trong ô dữ liệu"	Áp quy tắc CSV sang JSON (lỗi #2, Pass giả)	"...Nêu rõ quy tắc nào chỉ đúng cho CSV, quy tắc nào chỉ đúng cho JSON. Với mỗi phép biến đổi, tự hỏi: nếu test case đang kiểm CHÍNH phép biến đổi này thì loader có làm họ SUT không?"
"Sinh file dữ liệu cho feature X"	Ghi không dấu ở cột dữ liệu (lỗi #7)	"...Quy ước không dấu CHỈ áp dụng cho cột ghi chú. Cột nào có giá trị phải khớp chính xác với UI (nhân <select>, thông báo) thì viết đúng nguyên văn như UI hiển thị."
"Viết page object cho màn hình X"	Sai tên khoá so với schema (lỗi #8)	"...Tham số của hàm phải trùng TÊN CỘT trong file dữ liệu, không đặt tên theo thói quen. Liệt kê tên cột trước khi viết hàm."
"Viết test gọi API kiểm quyền truy cập"	Ảnh chụp Fail trắng (lỗi #9)	"...Test thuần API vẫn phải điều hướng page tới nơi thể hiện hậu quả, để ảnh chụp lúc Fail có giá trị làm bằng chứng."
"Sinh script chạy nhiều lượt"	Tự thêm cờ CLI phá config (lỗi #11)	"...Trước khi thêm bất kỳ cờ dòng lệnh nào, đọc lại comment trong file config xem cờ đó có bị cảnh báo không."

Nguyên tắc tổng quát rút ra: khoảng cách nguy hiểm nhất không nằm giữa *prompt* và *code AI sinh ra*, mà nằm giữa **ý định đã được viết thành lời** (comment, tên biến, mô tả TC) và **hành vi thật sự của code**. Lỗi #10 và #11 đều là dạng này: comment mô tả đúng, code làm khác. Cách duy nhất bắt được là **chạy thật rồi đối chiếu kết quả với ý định**, không phải đọc code.

4. Test case không automation được (§6 bắt buộc)

4.1 Test case của HW02 không đưa vào suite (8 TC)

Không TC nào bị bỏ vì khó automation. Cả 8 đều là **trùng lặp cơ chế** với một TC khác đã có trong suite, hoặc HW02 đã kết luận **không phải bug**:

TC (HW02)	Vì sao không đưa vào	TC thay thế trong suite
FR02-BV-04	Cùng kịch bản với DT-07 (nhập đúng mật khẩu khi đang khóa)	FR02-DT-07
FR02-BV-05	Mốc ~30s nằm trong cùng chuỗi đo với mốc 31s	FR02-BV-06
FR02-BV-07	Mốc 31s — đã gộp thành một TC <code>lockout-wait</code>	FR02-BV-06
FR02-BV-R01	Cùng cơ chế chặn (HTML5 required) với 2 TC đã có	FR02-DT-04, FR02-DT-06
FR02-BV-R02	HW02 đã kết luận không phải bug (spec không đòi trim email)	—
FR09-BV-07	Trùng hoàn toàn với DT-09 (VIP100 hết lượt)	FR09-DT-09
FR09-BV-R01	Trùng với DT-05 (mã rỗng → nút bị vô hiệu hoá)	FR09-DT-05
FR15-BV-01	Trùng với DT-05 (giá = 0)	FR15-DT-05

Đồng thời **bổ sung 5 TC mới ở HW04** mà HW02 không có: FR15-DT-09 , FR15-BV-R03 , FR15-SEC-01/02/03 — và chính 4 trong 5 TC này tìm ra **cả hai bug mới** của bài.

4.2 Phần của test case chỉ automation được một phần

TC	Phần không automation được	Vì sao	Cách xử lý thay thế
FR02-BV-06	Đo chính xác thời điểm hết khóa (~180s)	Chờ 3 phút × 3 engine = 9 phút chỉ để xác nhận lại điều mà mốc 31s đã chứng minh; chi phí thời gian không tương xứng giá trị thông tin	Assert tại 31s — spec đòi 30s nên vẫn còn khóa ở mốc này đã đủ kết luận vi phạm. Con số 180s giữ bằng chứng thủ công từ HW02 (B002-endafter3minutes.png)
FR09-DT-10	Phần "hoàn tất thanh toán tạo đơn hàng giả 350 triệu"	Đơn hàng ghi vào bảng orders, không có API xóa ⇒ 9 lượt chạy sẽ để lại 9 đơn rác, làm nhiễu dữ liệu của FR-18 và các HW sau	Automation tới bước áp mã + đọc số hiển thị + kiểm ô tổng tiền sửa được . Phần tạo đơn giữ bằng chứng thủ công HW02 (B013-1.png, B013-2.png)
FR15-BV-R03	Nhập <code>category_id</code> = 9999 qua giao diện	Ô danh mục là <code><select></code> chỉ liệt kê 3 danh mục seed ⇒ không thể nhập giá trị ngoài danh sách bằng thao tác người dùng	Hạ xuống tầng API với token hợp lệ — không bỏ TC. Chính cách này tìm ra BUG-NEW-02
FR15-SEC-01/02/03	Không thể chạm tới qua giao diện	Admin UI chặn user thường ngay tại màn đăng nhập (App.jsx:65)	Gọi thẳng API bằng context không có token — chính cách này tìm ra BUG-NEW-01

Hai dòng cuối cho thấy một điều đáng ghi: "không automation được qua UI" *không đồng nghĩa với "không automation được". Hạ xuống tầng API là cách xử lý đúng, và trong bài này nó chính là thứ tìm ra cả hai bug mới.

5. Bug phát hiện

Chi tiết đầy đủ ở [bug-report/bug-report.md](#) . Tóm tắt:

	Số defect
Bug MỚI phát hiện ở HW04	2
Defect đã ghi từ HW02, automation tái lập lại được	14
Tổng	16

Hai bug mới

Bug	Mức độ	TC phát hiện	GitHub Issue	Vì sao kiểm thử công không thấy
BUG-NEW-01 — POST/PUT/DELETE /api/products không yêu cầu token (vi phạm FR-12)	 Critical	FR15-SEC-01/02/03	#1	Admin UI chặn user thường trước khi chạm tới API. HW02 đã nghi ngờ khi đọc code (Main_Report.md dòng 468) nhưng ghi rõ " KHÔNG test được từ UI → không tạo TC"
BUG-NEW-02 — chấp nhận category_id không tồn tại	 High	FR15-BV-R03	#2	<select> chỉ có 3 danh mục seed, không nhập được giá trị ngoài danh sách qua UI

Cả hai Issue đã tạo trên repo bài làm, kèm ảnh bằng chứng nhúng trong phần mô tả; ảnh chụp trang Issue lưu ở `bug-report/screenshots/issue-1.png` và `issue-2.png` (§14).

Hai kết luận chặt hơn HW02 nhờ automation

- B007 nằm ở tầng tính toán của API, không phải tầng hiển thị.** Pattern P3 (đối chiếu số trên UI với body API thật) **không Fail ở bất kỳ TC nào** ⇒ UI hiển thị **đúng y nguyên** con số API trả về. Kiểm thử công chỉ nhìn thấy con số cuối trên màn hình nên không tách bạch được.
- B014 nằm ở client, không phải backend.** TC FR15-DT-08 assert **hai tầng**: UI hiện sai tên sản phẩm khác, nhưng DB vẫn đúng ⇒ lỗi ở `frontend-admin/src/App.jsx` (`setProducts(products.map(...))` sau khi PUT), không phải ở API.

6. Kết luận

6.1 Đối chiếu với yêu cầu định lượng của §6

Yêu cầu §6	Mức đạt
≥ 12 test case automation / feature	A: 16 · B: 18 · C: 19 (tổng 53)
Test data ở file .csv/.json ngoài script	 cả hai định dạng, grep xác nhận 0 hard-code

≥3 assertion pattern khác bản chất	5 pattern (P1–P5)
≥9 lượt browser, mỗi lượt 1 HTML report	✓ 9 lượt, 9 report độc lập
Report hiện Run by: {StudentID} + timestamp ISO	✓ ở 3 chỗ trên mỗi report
Human review + gap analysis	✓ 12 lỗi thật kèm nguyên nhân cụ thể (§3.2)
Bug → Markdown + GitHub Issue kèm ảnh	✓ 2 bug mới, ảnh có watermark MSSV
Ghi rõ TC không automation được và vì sao	✓ \$4

6.2 Giá trị của automation so với kiểm thử thủ công ở HW02

Bài này cho ba bằng chứng cụ thể, không phải nhận định chung chung:

1. **Tìm ra lỗ hổng bảo mật mà thủ công không chạm tới được.** HW02 đã **ngghi ngờ** BUG-NEW-01 khi đọc code nhưng buộc phải bỏ vì UI chặn đường. Automation gọi thẳng API thì vượt qua rào cản đó.
2. **Tách được tầng gây lỗi.** Assertion P3 (đối chiếu UI với DB/API) chứng minh B007 ở tầng API và B014 ở tầng client — kết luận mà mắt người nhìn màn hình không đưa ra được.
3. **Lập lại được 159 lần thực thi với 0 flaky.** Toàn bộ 14 bug đã tìm bằng tay ở HW02 đều được tái lập tự động, không sót cái nào.

6.3 Hạn chế còn lại

- 2 TC (FR09-DT-02/03) Fail trên firefox vì **lỗi hạ tầng của Firefox** lúc đóng browser context. Chưa tìm được cách khắc phục ở tầng script; đã xác minh không ảnh hưởng kết luận nghiệp vụ.
- Phần đo chính xác thời gian khóa 180s và phần tạo đơn hàng giả 350 triệu **cố ý** không automation (§4.2), giữ bằng chứng thủ công từ HW02.

6.4 Tự đánh giá (\$15)

No.	Tiêu chí	Điểm tối đa	Tự chấm	Căn cứ
1	Task 1 — Feature A (FR-02)	25	25	16 TC (yêu cầu ≥12) · dữ liệu ở .csv ngoài · dùng P1/P2/P3/P5 · cô lập bằng user dùng-một-lần nên 3 engine cho kết quả giống hệt · 10 TC Fail truy về đúng 6 bug đã biết
1	Task 1 — Feature B (FR-09)	25	25	18 TC · dữ liệu .json · phát hiện & siết 1 Pass giả (§3.3) · 2 TC đi luồng giỏ hàng thật · P3 chứng minh B007 ở tầng API
1	Task 1 — Feature C (FR-15)	25	25	19 TC · tìm ra 2 bug MỚI , trong đó 1 Critical (broken access control) mà HW02 xác nhận không test được từ UI · assert 2 tầng UI+DB cho kết luận chặt hơn HW02
2	Task 2 — Video demo	15	<ĐIỀN>	<ĐIỀN SAU KHI QUAY>
3	Agent Skills	10	<ĐIỀN>	<ĐIỀN SAU KHI LÀM docs/10>
	Tổng	100	<ĐIỀN>	

Phụ lục — cấu trúc bài nộp

tests/	data/ (3 file dữ liệu) · pages/ (5 page object) · utils/ · fixtures/ · 3 spec
tools/	preflight · run-all-browsers · stamp-report · summarize · capture-*-evidence
reports/	html/<f>-<e>/ ×9 · json/ ×9 · summary.md · evidence/ (19 ảnh)
report/	main-report.md (file này)
bug-report/	bug-report.md · issue-new-01.md · issue-new-02.md · screenshots/
ai-audit/	ai-audit-report.md · ai-critique.md
docs/	14 file hướng dẫn quy trình